

Guia de Preparação — Apresentação

ANN em Hardware

Documento de estudo para a apresentação. Não é para ler nos slides — é para você internalizar a narrativa e estar pronto para qualquer pergunta do teu mentor.

Tempo alvo: 10-12 minutos, ~1.5-2 min por slide. Prática recomendada: fala em voz alta pelo menos uma vez cronometrando.

Slide 1 — Fundamentos do Neurônio em Hardware

O que dizer

Comece pela equação: um neurônio artificial é, na sua essência, uma soma ponderada seguida de uma não-linearidade — $y = f(\sum x \cdot w + b)$. Em hardware, isso vira um circuito **MAC** (Multiply-Accumulate): 4 multiplicadores, um somador, e um bloco de ativação no fim.

A decisão de design mais fundamental foi usar **ponto fixo Q8.8** em vez de ponto flutuante. Justifique com o trade-off: ponto flutuante (IEEE 754) precisaria de lógica muito mais complexa (expoente, mantissa, normalização) — cara em área e lenta. Q8.8 é 16 bits fixos: 8 inteiros + 8 fracionários, escala 256. 1.0 decimal vira $256 = 0 \times 0100$.

Números-chave para ter na ponta da língua

- Escala Q8.8 = 256
- $1.0 \rightarrow 0 \times 0100$, $0.5 \rightarrow 0 \times 0080$
- Representação com sinal: complemento de dois

Perguntas prováveis

- **"Por que não ponto flutuante?"** → Área e velocidade. Ponto fixo é só inteiros com escala implícita — soma e multiplicação são operações nativas de ALU, sem necessidade de normalização/expoentes.
 - **"Por que 8+8 e não, por exemplo, 4+12?"** → Compromisso: 8 bits fracionários dão resolução de $\sim 1/256$ ($\sim 0.4\%$), suficiente para os testes feitos; 8 bits inteiros cobrem a faixa de valores esperada (± 128) sem overflow nos casos testados.
 - **"O que acontece se um valor ultrapassar a faixa representável?"** → É exatamente a limitação que discutimos no Slide 7 (trabalho futuro): há risco de wrap-around no truncamento dos produtos, sem proteção de saturação implementada.
-

Slide 2 — Arquitetura RTL e Pipeline Síncrono

O que dizer

Mostre os dois pipelines lado a lado. A ReLU tem 3 estágios de registo; a Sigmoid tem 4 (o estágio extra vem da leitura da ROM, que também é síncrona). Aponte visualmente as barreiras `clk` entre os blocos — cada barreira é uma borda de subida do relógio.

Depois, conte o **caso de estudo do bias** como uma história real de debugging, não como teoria abstrata — isso é o que mais impressiona um mentor técnico:

1. Percebemos que os produtos ($x \cdot w$) só ficam prontos 2 ciclos depois da entrada (1 ciclo para capturar x/w , mais 1 ciclo para registar o produto).
2. Se o bias fosse somado direto na entrada (sem atraso), ele chegaria "adiantado" — no regime de streaming contínuo (uma amostra nova por ciclo), isso mistura o bias de uma amostra com os produtos de outra.
3. A correção: adicionar um segundo registo (`b_reg2`) para o bias, e o mesmo princípio aplicado aos flags de saturação da Sigmoid (`sat_low_reg2`, `sat_high_reg2`).
4. Valide dizendo que isso foi **confirmado experimentalmente**: aplicamos vetores com bias diferentes em ciclos consecutivos na simulação, e a saída de cada amostra usou exatamente o bias correto.

Números-chave

- ReLU: 3 ciclos | Sigmoid: 4 ciclos
- `b_reg` → `b_reg2` (2 estágios)
- `sat_low_reg/sat_high_reg` → `sat_low_reg2/sat_high_reg2` (registados duas vezes na Sigmoid)

Perguntas prováveis

- "Por que a Sigmoid precisa de um estágio a mais?" → Porque a `Sigmoid_ROM` tem seu próprio registo de saída síncrono (leitura de memória registada = boa prática para inferir BRAM/LUTRAM eficientemente na FPGA). Isso soma +1 ciclo ao pipeline total.
 - "Essa correção do bias era mesmo necessária, ou só teórica?" → Só é estritamente necessária em **streaming contínuo** (nova amostra a cada ciclo). Se o sistema processasse uma amostra de cada vez, esperando o resultado sair antes da próxima entrada, não haveria erro — mas como não há sinais `valid/ready` no design, assumimos o cenário mais exigente (streaming) por segurança e para não limitar o throughput futuro.
 - "Isso não aumenta a latência desnecessariamente?" → Latência aumenta em 0 ciclos adicionais além do que a ROM já impõe — os registos extras do bias/flags rodam **em paralelo** com os estágios já existentes, não adicionam ciclos novos ao pipeline, só garantem que os dados certos se encontram no momento certo.
-

Slide 3 — ReLU vs. Sigmoid: Estratégias de Ativação

O que dizer

Mostre as duas curvas reais (plotadas matematicamente, não são ícones genéricos). ReLU é "quase de graça" em hardware: só precisa olhar o bit de sinal do resultado e escolher entre o valor ou zero — um multiplexador 2:1. A Sigmoid, sendo uma curva suave e não-linear, não dá para calcular "ao vivo" (exigiria exponenciação em hardware, proibitivamente caro) — por isso usamos uma **tabela pré-calculada (ROM)**.

Explique o truque do endereçamento: mapeamos o intervalo $[-4.0, +4.0]$ (onde a sigmoide já cobre ~98% da sua variação útil) para 256 endereços, com a fórmula $\text{addr} = (\text{soma_total} + 1024) \gg 3$. Fora desse intervalo, a função já está praticamente saturada (perto de 0 ou 1), então forçamos a saída diretamente, sem consultar a ROM.

Números-chave

- $\sigma(4) \approx 0.982, \sigma(-4) \approx 0.018$ (por isso o intervalo $[-4, +4]$)
- $x=-4.0 \rightarrow \text{addr}=0 \mid x=0.0 \rightarrow \text{addr}=128 \mid x \approx +4.0 \rightarrow \text{addr}=255$
- Saturação: $\text{soma} \leq -1024 \rightarrow y=0x0000 \mid \text{soma} \geq +1024 \rightarrow y=0x0100$

Perguntas prováveis

- "Por que 256 posições e não mais/menos?" $\rightarrow$ 8 bits de endereço = 256 posições é um bom equilíbrio entre resolução (erro de quantização pequeno) e tamanho de tabela (mantém a ROM pequena o suficiente para virar lógica distribuída em vez de precisar de BRAM dedicada).
- "Como os valores da ROM foram gerados?" $\rightarrow$ Script Python que calcula a sigmoide no **centro de cada faixa de 8 valores** representada por um endereço (não na borda), minimizando o erro médio de quantização, depois converte para Q8.8.
- "Essa aproximação por tabela introduz que tipo de erro?" $\rightarrow$ Erro de quantização (resolução limitada a 256 pontos) e erro de truncamento (Q8.8 tem só 8 bits fracionários) — ambos pequenos o suficiente para os propósitos do projeto, mas mencionáveis se perguntado sobre precisão.

Slide 4 — Verificação Funcional em Streaming Contínuo

O que dizer

Esse slide é a prova de que tudo funciona antes de fisicamente ir para o silício. Mostre as waveforms reais do Vivado — são capturas de tela genuínas da tua simulação, não ilustrações. Aponte que a latência aparece **fisicamente** na waveform: a entrada muda, e a saída só acompanha 3 (ReLU) ou 4 (Sigmoid) bordas de clock depois.

O ponto mais forte para enfatizar aqui: você não testou só "será que dá a resposta certa parada" — testou em **regime de streaming**, aplicando vetores diferentes em ciclos consecutivos, incluindo casos desenhados especificamente para expor o bug de desalinhamento do bias (se ele existisse). Isso é rigor de verificação de verdade, não só "rodei uma vez e funcionou".

Números-chave

- 8 vetores de teste: 4 "fáceis" (conferíveis à mão) + 4 "aleatórios"
- Todos bateram exatos com o cálculo manual (ex.: V2 ReLU deu 0×0100 exato para 1.0×1.0)
- Testbench gerado em SystemVerilog, clock de 10ns (100MHz) na simulação

Perguntas prováveis

- "Como você sabe que a simulação está correta, e não só 'parece' certa?" → Cálculo manual prévio dos valores esperados (multiplicação e soma em Q8.8), comparados byte a byte com a saída do simulador — não foi confiança visual, foi conferência numérica.
- "O testbench cobre casos de erro/borda?" → Sim: vetores desenhados para forçar saturação (positiva e negativa), soma exatamente zero, e a troca de bias entre ciclos consecutivos (teste de streaming).
- "Por que 8 vetores e não mais?" → Equilíbrio entre cobertura (casos triviais + extremos + aleatórios) e tempo de simulação/análise — suficiente para validar a lógica sem virar um exercício de força bruta.

Slide 5 — Resultados de Síntese (Tabela de Recursos)

O que dizer

Aqui os dados "descem para o silício de verdade": não é mais RTL abstrato, é o que o Vivado realmente colocou na FPGA Artix-7 depois de posicionar e rotear. Destaque o achado mais interessante: **DSPs idênticos (40) nas duas versões** — prova de que a mudança entre ReLU e Sigmoide ficou isolada só no estágio final de ativação, sem afetar o bloco MAC.

O segundo ponto forte: a ROM da Sigmoide, apesar de ter 256×16 bits, **não usou nenhum Block RAM** — o Vivado a implementou como uma árvore de multiplexadores (MUXF7/MUXF8) alimentada por LUTs, porque é pequena e somente-leitura. Isso confirma a decisão de replicar a ROM em cada neurônio em vez de centralizar (10 ROMs pequenas, sem necessidade de arbitragem de acesso).

Números-chave (decore estes)

	ReLU	Sigmoide
LUTs	680 (3.27%)	1090 (5.24%)

	ReLU	Sigmoide
FFs	1280 (3.08%)	1320 (3.17%)
DSPs	40 (44.44%)	40 (44.44%)
BRAM	0	0

Perguntas prováveis

- **"Por que modo Out-of-Context?"** → O `Hidden_Layer` como bloco IP tem 1026 portas de I/O (10 neurônios × pesos/bias/saída de 16 bits) — muito acima dos 106 pinos físicos da FPGA. OOC ignora a restrição de mapear cada porta num pino físico, tratando o design como um bloco reutilizável que, num sistema real, receberia dados de memória interna, não de pinos externos.
- **"Por que essa FPGA específica (Artix-7 xc7a35t)?"** → É a FPGA da placa Basys 3, comum em contexto acadêmico — familiar e com relatórios fáceis de interpretar/comparar.
- **"+40 Flip-Flops na Sigmoide parece pouco, dado que ela tem 4 registros a mais por neurônio (addr, 2 flags×2). Por quê?"** → Provável otimização do Vivado por *constant propagation*: como a saída da sigmoide está sempre em $[0 \times 0000, 0 \times 0100]$, vários bits altos são quase sempre constantes e podem ter sido simplificados/eliminados na síntese.

Slide 6 — Timing e Caminho Crítico

O que dizer

Aqui você mostra que entende timing de verdade, não só "rodei e não deu erro". Explique o conceito de **WNS** (Worst Negative Slack) — a folga de tempo no pior caminho do circuito — e como isso converte em $F_{\max}$: $F_{\max} = 1 / (\text{Período} - \text{WNS})$.

O achado mais valioso deste slide: a queda de $F_{\max}$ entre ReLU e Sigmoide é pequena (só ~3%), **apesar de** a Sigmoide ter 60% mais LUTs. Isso parece contraintuitivo até você mostrar o porquê: o caminho crítico da Sigmoide **não passa pela ROM** (que é rápida, só 1 nível de lógica de seleção) — ele passa pela árvore de comparação dos flags de saturação, que tem 10 níveis de lógica. Ou seja: o "custo real" em velocidade não vem da memória, vem da lógica de decisão ao redor dela. Esse é o tipo de insight que mostra análise de engenharia real, não só leitura de números de um relatório.

Números-chave

- $F_{\max} \text{ ReLU} \approx 132.7 \text{ MHz}$ (WNS = 2.465ns)
- $F_{\max} \text{ Sigmoide} \approx 128.75 \text{ MHz}$ (WNS = 2.232ns)
- Período de referência usado na constraint: 10ns (100MHz)
- Caminho crítico da Sigmoide: 7.534ns total (43.9% lógica / 56.1% roteamento), 10 níveis de lógica (6×CARRY4 + LUT2 + 2×LUT5 + LUT6)

Perguntas prováveis

- **"Por que usar 100MHz como referência se a FPGA aguenta mais?"** → É só um "alvo" inicial para o Vivado calcular contra — o número real (Fmax) é derivado da folga (WNS) sobrando desse alvo. Poderíamos ter usado qualquer período de referência; o resultado do Fmax real seria o mesmo.
- **"Todos os caminhos passaram (sem violação)?"** → Sim, WNS positivo em ambas as versões — nenhuma violação de Setup/Hold, incluindo nos caminhos que envolvem os registos extras de alinhamento do bias/flags.
- **"Isso significa que a ROM é 'grátis' em termos de velocidade?"** → Não seria correto dizer "grátis" — ela adiciona lógica (os multiplexadores) e um ciclo de latência — mas o **caminho crítico específico** não passa por ela. Outro caminho, em outro cenário de design, poderia sim ser limitado pela ROM.

Slide 7 — Escalabilidade e Trabalho Futuro

O que dizer

Feche com uma reflexão de arquitetura mais ampla: com só 10 neurônios de 1 hidden layer, **já ocupamos 44% dos DSPs** desta FPGA pequena. Isso não é um problema do design atual — é uma limitação fundamental do paralelismo puro (1 MAC dedicado por neurônio). Se o projeto crescesse (mais neurônios, mais camadas), a solução não seria "otimizar" o design atual, seria mudar de arquitetura: **time-multiplexing** — reutilizar um número menor de multiplicadores físicos, processando os neurônios em série ao longo de vários ciclos, em vez de todos em paralelo.

Termine com os dois itens de trabalho futuro, apresentados como decisões conscientes de escopo (não como falhas):

1. Proteção de saturação contra wrap-around no truncamento dos produtos — hoje ausente, mencionável como próximo passo de robustez.
2. Paridade completa do fluxo de verificação (simulação + síntese) para a versão VHDL — hoje só validamos formalmente a versão SystemVerilog, embora o código VHDL tenha paridade estrutural completa.

Números-chave

- DSPs: 44.44% ocupados (40 de 90 disponíveis na xc7a35t)
- LUTs/FFs bem abaixo de 10% — não são o gargalo

Perguntas prováveis

- **"Time-multiplexing não pioraria a latência?"** → Sim, seria um trade-off consciente: menos área (menos multiplicadores físicos) em troca de mais ciclos por amostra processada — útil quando o gargalo é área/DSPs disponíveis, não velocidade de resposta.
- **"Por que não implementaram a saturação contra overflow desde já?"** → Escolha de escopo: o foco do projeto foi a arquitetura de pipeline e o mecanismo de ativação; a robustez contra overflow extremo é uma melhoria incremental que

não muda a validade dos resultados atuais (os vetores de teste usados não geraram overflow).

- **"O código VHDL foi testado?"** → Tem paridade estrutural e sintática completa com o SystemVerilog (mesma lógica, mesmos módulos), mas a bateria formal de testes (simulação + síntese + timing) documentada neste projeto foi conduzida na versão SystemVerilog.

Perguntas gerais que podem vir a qualquer momento

- **"Qual foi o maior desafio técnico do projeto?"** → O alinhamento temporal do bias/flags em streaming — não é óbvio até você simular com múltiplas amostras em sequência; é o tipo de bug que só aparece sob uso realista, não em testes triviais.
- **"O que você faria diferente se recomeçasse?"** → Provavelmente desenharia um sinal `valid/ready` desde o início, para tornar explícito o contrato de streaming, em vez de assumir implicitamente.
- **"Isso escalaria para uma rede maior (mais camadas)?"** → Estruturalmente sim (é só instanciar mais `Hidden_Layers`), mas os recursos de DSP se esgotariam rápido — é exatamente o ponto do Slide 7.

Dicas finais de apresentação

- Tenha a tabela do Slide 5 e os números de F_{max} do Slide 6 **memorizados** — são os números mais prováveis de gerar perguntas de acompanhamento.
- Se travessem em alguma pergunta técnica, é perfeitamente aceitável dizer "essa é uma boa consideração para trabalho futuro" e redirecionar para o Slide 7 — mostra maturidade de engenharia, não fraqueza.
- Pratica a transição entre Slides 5→6→7 em voz alta pelo menos uma vez — são os três slides mais densos em números, e a fluência nessa sequência é o que mais impressiona.